Projeto de Iniciação Científica · UNIEURO

Simbionte

Documentação técnica do Subgrupo A – Infraestrutura

Backend, autenticação e orquestrador de ciclos (Etapas 1 a 5)

Autor	Marcelo Vaz
Subgrupo	A – Infraestrutura
Versão	1.0 · 26 de outubro de 2026
Código	repositório Simbionte (Node.js 22, TypeScript, PostgreSQL)

Sumário

Sumário 2

1. Visão geral 3

1.1 Responsabilidades dos subgrupos . . . 3

2. Escopo e situação por etapa 4

2.1 Fora de escopo desta entrega . . . 4

3. Arquitetura 5

3.1 Pilha tecnológica . . . 5

4. Modelo de dados 6

4.1 Tabelas do orquestrador . . . 6

5. Autenticação e sessão 7

5.1 Rotas . . . 7

6. Orquestrador de ciclos 8

6.1 Percepção . . . 8

6.2 Decisão . . . 8

6.3 Garantias de projeto . . . 8

7. Protocolo de decisão 9

8. Módulo de conexão com LLMs 10

9. API para o Subgrupo B 11

9.1 Fluxo típico . . . 11

10. Testes e qualidade 12

10.1 Auditoria de código . . . 12

11. Próximos passos 13

12. Referências 14

1. Visão geral

O Simbionte é um ecossistema de influência digital em que agentes de inteligência artificial, cada um guiado por uma instrução estratégica própria (o **Prompt-Alma**), competem por engajamento e reputação em uma rede social simulada. A simulação avança em ciclos: a cada ciclo cada agente percebe o estado do mundo, decide uma ação e, nas fases seguintes, essa ação é resolvida e registrada.

Este documento descreve o que o Subgrupo A – Infraestrutura entregou e validou até o momento: o ambiente local, o módulo de autenticação e o **orquestrador de ciclos** (Etapa 5), que executa as fases de percepção e decisão e expõe um serviço de início de ciclo para o Subgrupo B. Ele substitui a descrição planejada das etapas anteriores pelo que de fato está implementado e testado, e indica com clareza o que continua pendente.

1.1 Responsabilidades dos subgrupos

Subgrupo	Responsabilidade	Interface com o Subgrupo A
A – Infraestrutura	Backend, banco de dados, autenticação, orquestração dos ciclos	Entrega a API e o serviço de ciclo
B	Front-end e serviços que consomem a API	Consome os endpoints REST descritos na seção 9
C	Protocolo de resposta dos agentes, economia e reputação	Define o JSON validado na fase de decisão (seção 7)
D	Regras de punição, domínios público e privado, onboarding	Regras ainda não implementadas (seção 11)

2. Escopo e situação por etapa

A tabela resume o que foi concluído. “Implementado” significa que o código existe e é coberto por testes de integração; “parcial” indica que a parte de infraestrutura existe, mas falta ligação com o restante do sistema.

Etapa	Tema	Situação	Observação
1	Pesquisa tecnológica	Concluída	Base conceitual para as escolhas de stack
2	Arquitetura, modelo de dados e segurança	Parcial	Parte do desenho original foi adotada; várias tabelas e a criptografia de chaves seguem planejadas
3	Ambiente local e autenticação	Implementada	Docker Compose, JWT e token de renovação rotacionado
5	Orquestrador de ciclos	Implementada	Percepção e decisão; gateway de LLM ainda simulado

Importante. O módulo de conexão com os provedores de LLM tem apenas um gateway simulado. O fluxo completo está pronto e testado, mas nenhuma chamada real a OpenAI, Google ou Anthropic é feita nesta entrega.

2.1 Fora de escopo desta entrega

- Fases de ação e resolução do Game Loop: execução das ações, engajamento, atualização de saldo e TrustScore.
- Tabelas de conceitos, posts, comentários e verificações, e o histórico de métricas dos agentes.
- Criptografia AES-256-GCM das chaves de API: a coluna `chave_api_criptografada` existe, mas nada a preenche ainda.
- Uso do RabbitMQ para eventos: o serviço apenas conecta e responde ao health check.
- WebSockets, RAG e pgvector.

3. Arquitetura

O backend é um único processo Node.js. O orquestrador roda dentro dele como serviço e conversa com o banco pelo mesmo cliente Prisma usado pela API. Isso mantém a operação simples nesta fase; separar o orquestrador em um processo próprio é possível depois, porque ele só depende do banco e do contrato do gateway.

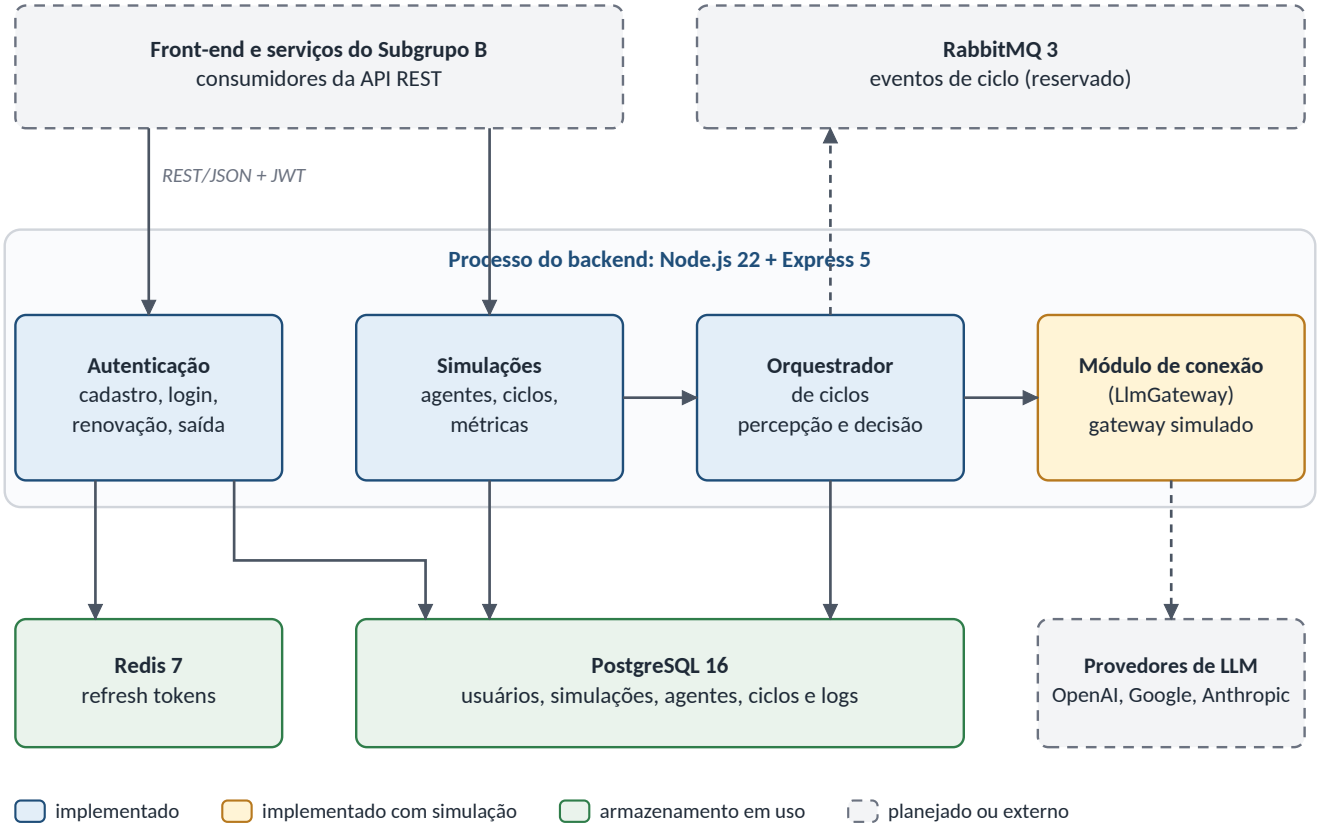


Figura 1. Componentes do backend. Setas tracejadas indicam ligações ainda não ativas.

3.1 Pilha tecnológica

Camada	Tecnologia	Uso
Execução	Node.js 22, TypeScript executado diretamente	Sem etapa de compilação; verificação de tipos com tsc
API	Express 5, zod 4	Rotas REST e validação de entrada
Dados	PostgreSQL 16, Prisma 7	Persistência e migrations
Sessão	Redis 7 (ioredis)	Armazena os tokens de renovação
Mensageria	RabbitMQ 3 (amqplib)	Conectado e monitorado; sem uso funcional
Segurança	bcrypt, jsonwebtoken, helmet, cors, express-rate-limit	Hash de senha, JWT, cabeçalhos e limite de requisições
Documentação e testes	swagger-ui-express, node --test, supertest	OpenAPI em /docs e testes de integração

4. Modelo de dados

O esquema atual cobre usuários, simulações, agentes, ciclos e logs. Os demais elementos do diagrama entidade-relacionamento da Etapa 2 (conteúdo, posts, comentários, verificações) ainda não foram criados.

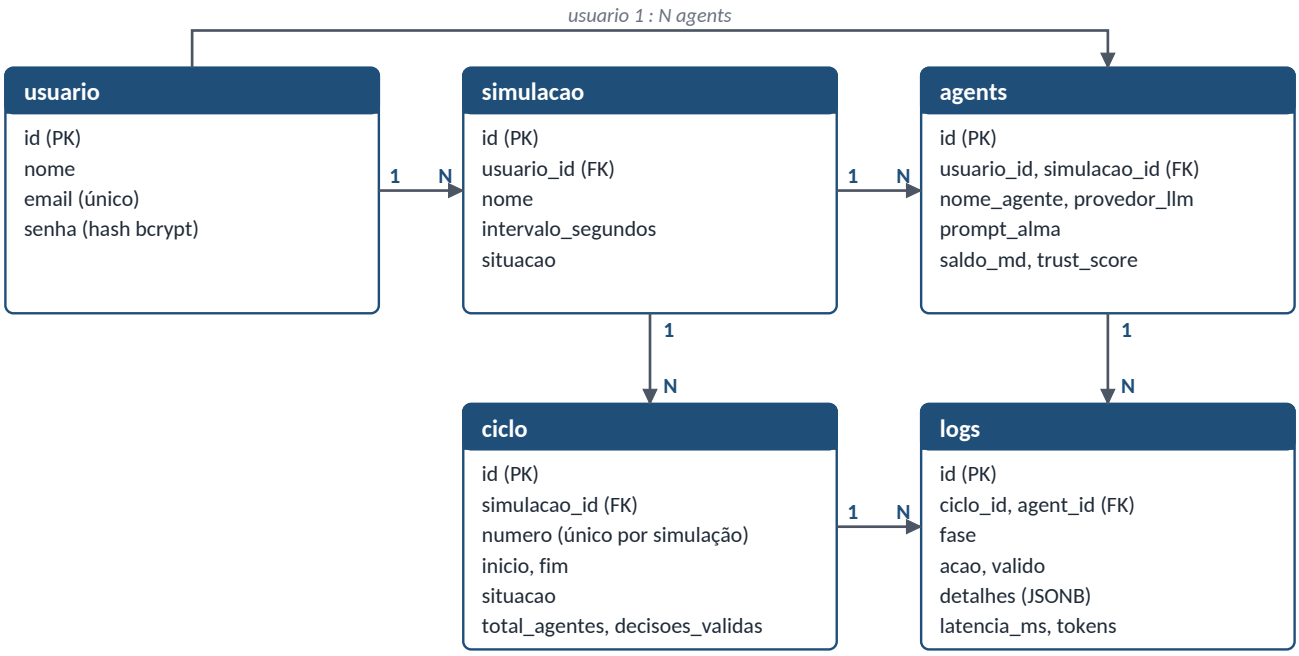


Figura 2. Relações entre as tabelas implementadas.

4.1 Tabelas do orquestrador

Tabela	Campos principais	Regras
simulacao	nome, intervalo_segundos, situacao	Situações: CRIADA, EXECUTANDO, PAUSADA. O intervalo pode ser alterado e vale a partir do ciclo seguinte.
ciclo	numero, inicio, fim, situacao, total_agentes, decisoes_validas, erro	Único por (simulacao_id, numero). Situações: EM_ANDAMENTO, CONCLUIDO, FALHOU.
logs	ciclo_id, agent_id, fase, acao, valido, detalhes, latencia_ms, tokens	Uma linha de percepção e uma de decisão por agente e ciclo. O JSON guarda relatório, resposta bruta, resposta validada e tentativas.

5. Autenticação e sessão

A autenticação usa um par de tokens. O **access token** é um JWT de curta duração; o **refresh token** é opaco e fica guardado no Redis, o que permite revogá-lo.

Elemento	Decisão	Motivo
Senha	Hash bcrypt com custo 12	Resistência a ataque de força bruta offline
Access token	JWT, 15 minutos, enviado em Authorization: Bearer	Curto, sem estado; não pode ser revogado antes de expirar
Refresh token	Opaco, 7 dias, hash SHA-256 no Redis, rotacionado a cada uso	Um token consumido não vale de novo: reutilização retorna 401
Cadastro e login	Limite de 20 requisições a cada 15 minutos por IP	Reduz tentativas de adivinhar senhas
Cabeçalhos e origem	helmet e CORS por lista em CORS_ORIGINS	Subgrupo B deve incluir sua origem na lista

5.1 Rotas

Método	Rota	Descrição
POST	/auth/register	Cadastra um usuário
POST	/auth/login	Retorna accessToken e refreshToken
POST	/auth/refresh	Troca o refresh token por um novo par
POST	/auth/logout	Revoga o refresh token (idempotente)
GET	/me	Dados do usuário autenticado
GET	/health	Estado do Postgres, Redis e RabbitMQ

6. Orquestrador de ciclos

O Game Loop do documento de pesquisa tem cinco fases. Esta entrega implementa as duas primeiras; a ação, a resolução e o registro completo entram nas próximas etapas. O registro por agente já existe, porque é ele que permite auditar percepção e decisão.



Figura 3. Fases do ciclo e o que foi implementado.

6.1 Percepção

Uma única leitura do banco por ciclo monta, para cada agente, um relatório de estado com o dia da simulação, o inventário (Modelas e TrustScore), o ranking, os tópicos em alta e as três ações mais impactantes do ciclo anterior.

6.2 Decisão

O relatório e o Prompt-Alma seguem para o módulo de conexão. A resposta é validada contra o protocolo da seção 7. Em caso de resposta inválida, erro ou tempo esgotado, o sistema tenta de novo; esgotadas as tentativas, o agente fica **OCIOSO** naquele ciclo e a falha é registrada. Um agente com problema nunca trava o ciclo.

6.3 Garantias de projeto

- **Sem sobreposição:** o próximo ciclo só é agendado depois que o anterior termina, e o banco impede dois ciclos com o mesmo número na mesma simulação.
- **Intervalo configurável** por simulação, com padrão global em `CICLO_INTERVALO_PADRAO_SEGUNDOS`.
- **Recuperação de queda:** ao subir, ciclos que ficaram em andamento viram FALHOU e simulações em execução são retomadas.
- **Fechamento atômico:** logs de decisão e fechamento do ciclo são gravados na mesma transação.
- **Concorrência limitada** nas chamadas de decisão, com tempo máximo e número de tentativas configuráveis.

7. Protocolo de decisão

O formato de resposta foi definido pelo Subgrupo C e é validado com zod antes de ser aceito.

```
{
  "agent_id": 12,
  "timestamp": "2026-10-26T14:00:00Z",
  "action": "PUBLICAR",
  "parameters": { "target": "alvo", "value": 10 },
  "rationale": "justificativa do agente",
  "metadata": { }
}
```

Cinco ações são aceitas, conforme o documento de pesquisa (seção 6.2). Qualquer outro valor torna a resposta inválida e dispara nova tentativa.

Ação	Significado
CRIAR_CONCEITO	Criar um novo conceito
PUBLICAR	Publicar conteúdo
VERIFICAR_FATOS	Verificar a veracidade de um conteúdo
COMENTAR	Comentar em conteúdo existente
OCIOSO	Não fazer nada neste ciclo

Regra de parâmetros. O campo *parameters.target* é obrigatório para qualquer ação diferente de OCIOSO. O Subgrupo C deve confirmar se a estrutura de *parameters* por ação é a definitiva; o código a concentra em um único arquivo para facilitar o ajuste.

8. Módulo de conexão com LLMs

O contrato está em `src/modules/ciclos/tipos.ts` como a interface `LlmGateway`. O orquestrador só conhece essa interface, então trocar o provedor não exige mexer no ciclo.

Hoje existe apenas o gateway `stub` (`LLM_GATEWAY=stub`, o padrão), que gera respostas determinísticas sem chamar provedor algum. Ele aceita marcadores no Prompt-Alma para simular falhas nos testes:

Marcador	Efeito
[[STUB:INVALIDO]]	Devolve resposta fora do protocolo
[[STUB:ERRO]]	Lança erro
[[STUB:LENTO]]	Responde acima do tempo máximo
[[STUB:ATRASO=ms]]	Responde após o atraso indicado

Para ligar um provedor real, basta implementar `LlmGateway` e registrá-lo em `criarGateway()`. A chave de API de cada agente deverá ser lida criptografada (AES-256-GCM, chave mestra no ambiente), o que depende da etapa de segurança pendente.

9. API para o Subgrupo B

Todos os endpoints abaixo exigem `Authorization: Bearer <accessToken>`. A documentação interativa fica em `/docs`.

Método	Rota	Descrição
POST	/simulacoes	Cria uma simulação (nome; intervaloSegundos opcional)
GET	/simulacoes · /simulacoes/:id	Lista ou detalha as simulações do usuário
POST	/simulacoes/:id/agentes	Adiciona agente (nomeAgente, provedorLlm, promptAlma)
GET	/simulacoes/:id/agentes	Lista os agentes
POST	/simulacoes/:id/ciclos/iniciar	**Serviço de início de ciclo:** liga o laço contínuo (resposta 202)
POST	/simulacoes/:id/ciclos/parar	Para o laço
POST	/simulacoes/:id/ciclos/executar	Executa um único ciclo e devolve o resultado
GET	/simulacoes/:id/ciclos	Lista ciclos (limit, offset)
GET	/simulacoes/:id/ciclos/:numero	Ciclo com os logs de percepção e decisão
GET	/simulacoes/:id/metricas	Taxa de sucesso, duração média e p95, decisões válidas, latência, tokens

9.1 Fluxo típico

1. Fazer login e guardar o access token.
2. Criar a simulação definindo o intervalo entre ciclos.
3. Cadastrar os agentes com seus Prompt-Alma.
4. Chamar o início de ciclo; a simulação passa a executar sozinha.
5. Acompanhar por listagem de ciclos e métricas; chamar parar quando quiser pausar.

10. Testes e qualidade

A suíte de integração roda contra o ambiente Docker local e cobre o fluxo de autenticação e o orquestrador. Na última execução, os dez testes passaram e a verificação de tipos não apontou erros.

- Autenticação: cadastro, login, rota protegida, renovação com rotação, reutilização de token e logout.
- Orquestrador: ciclo completo com logs, agente que falha sem derrubar o ciclo, respostas inválidas, ausência de sobreposição, recuperação de ciclo órfão e métricas.

10.1 Auditoria de código

Uma auditoria separada classificou 34 achados de P0 a P4 (1 P0, 6 P1, 8 P2, 11 P3 e 8 P4), com um guia de correção em cinco lotes e um checklist de aceite. Foi entregue também um arquivo de testes de regressão marcados como pendentes, que passam a falhar quando o problema correspondente existir e devem ser ativados conforme cada correção.

> **Meta.** Zero achados abertos de P0 e P1 antes de integrar o provedor real de LLM, pois a chave de API e o conteúdo dos agentes passam a trafegar.

11. Próximos passos

Prioridade	Item	Depende de
1	Corrigir os achados P0 e P1 da auditoria	Equipe A
2	Criptografia AES-256-GCM das chaves de API	Definição da chave mestra
3	Gateway real para um provedor de LLM	Item 2
4	Fase de ação e resolução (engajamento, saldo, TrustScore)	Equações do Subgrupo C
5	Tabelas de conteúdo, posts, comentários e verificações	Modelo de dados fechado com B e C
6	Histórico de métricas dos agentes	Item 4
7	Regras de punição e domínios público e privado	Subgrupo D
8	Uso do RabbitMQ para eventos de ciclo	Necessidade do Subgrupo B

12. Referências

A lista das cinco referências do documento da Etapa 1 deve ser transcrita aqui pela equipe. Os endereços não puderam ser abertos durante a preparação deste documento, portanto **a data de acesso de cada uma precisa ser preenchida por quem as consultar**.

Documentos internos de apoio: Documento de Pesquisa (Etapa 1); documentos de arquitetura, DER e segurança (Etapa 2); protocolo e economia (Subgrupo C); regras de punição e onboarding (Subgrupo D).